UNIT - 1

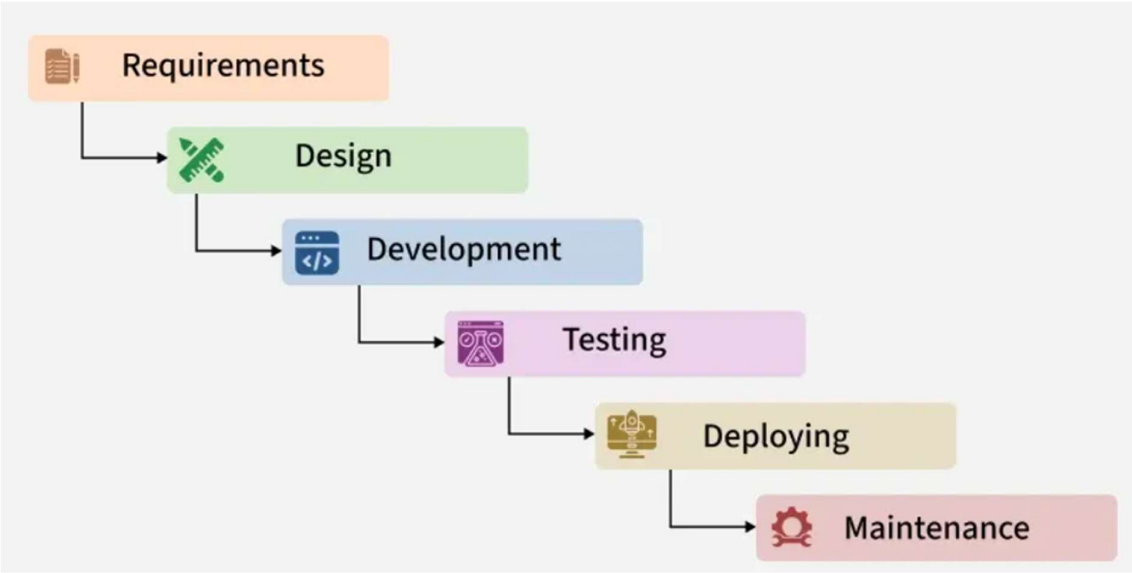
1. Waterfall Model:

Waterfall model provides a systematic, sequential approach to software development.

The waterfall model is also called as linear sequential model.

The software development starts with requirements gathering phase.

Then progresses through analysis, design, coding, testing and maintenance.



Waterfall Model

(i) Requirements gathering and Analysis:

In this phase, the basic requirements of the system must be understood by the software engineer.

All these requirements are documented for further use

(ii) Design: In this phase, the requirements are translated into easier format, using which coding can be done efficiently. The design focuses on program attributes such as, Data structure

- Software architectureeducational
- Interface representation
- Algorithmic details

(iii) Coding: The next phase is coding, in which the design is translated into machine-readable form Programs are created in this phase.

(iv) Testing: The purpose of testing is to uncover errors, fix the bugs and meet the customer requirements. Monal institute the testing ensures execution of all the paths and functional behaviours.

(v) Maintenance:

Maintenance is the longest life cycle phase.

When the system is installed and put into practical use, some errors may get introduced.

Correcting such errors is the major purpose of maintenance activity.

---

2. RAD Model:

- The Rapid Application Development (RAD) model is a type of incremental process model in which there is extremely short development cycle.
- Using the RAD model, the fully functional software system can be developed within 60 to 90 days.
- The various phases in RAD model are

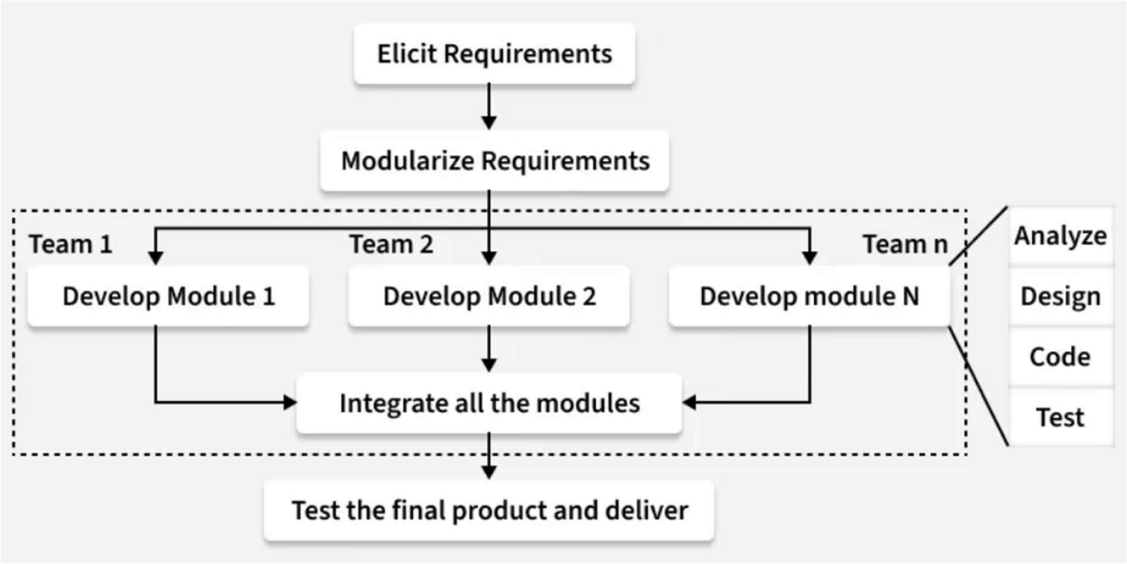
  (i) Requirements gathering

  (ii) Analysis and planning

  (iii) Design

  (iv) Build

  (v) Deployment



RAD Model

(i) Requirements gathering:

In this phase, the basic requirements of the system must be understood by the software engineer.

All these requirements are documented for further use.

(ii) Analysis and Planning:

During analysis and planning phase, the analysis on the gathered requirements is made.

Planning for various software development activities is done

(iii) Design:

During the design phase, various models such as business model, data model and process model are created.

(iv) Build:

During the build phase, the implementation code for the software system is created.

The functionalities developed by all the teams are integrated to form a whole system.

(v) Deployment:

Finally, the software system created by integrating all the software components, is deployed in the real world.

---

3. Spiral Model:

- Spiral model is a type of evolutionary process model.
- It possesses the iterative nature of prototyping model and the systematic approaches of the linear sequential model.
- The spiral model is divided into a number of framework activities, which are denoted by task regions. Usually, there are six task regions in the spiral model. They are,

  (i) Customer communication

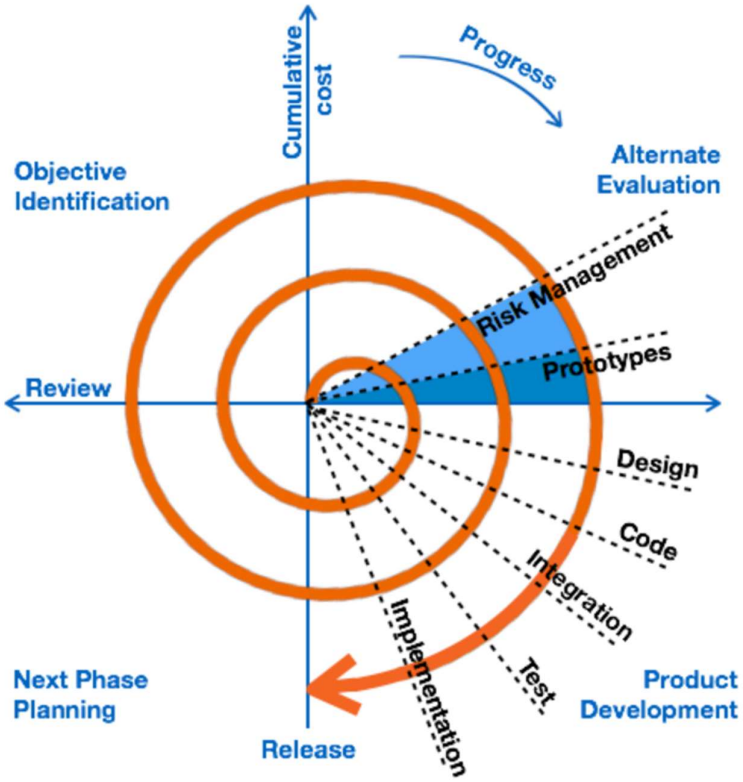
  (ii) Planning

  (iii) Risk analysis

  (iv) Engineering

  (v) Construct and Release

  (vi) Customer evaluation



(i) Customer communication:

In this region, customer communication about the project is established.

(ii) Planning:

In this region, the planning is done in order to define the resources timeline and other project related activities.

(iii) Risk analysis:

The tasks required to calculate the technical and management risks are carried out in this region.

(iv) Engineering:

In this task region, the tasks required to build one or more representations of the applications are carried out.

(v) Construct and Release:

All the tasks required to construct, test and install the application are carried out in this task region.

(vi) Customer evaluation:

In this region, the customer feedback is obtained. Based on the feedback, the required tasks are implemented the installation stage.

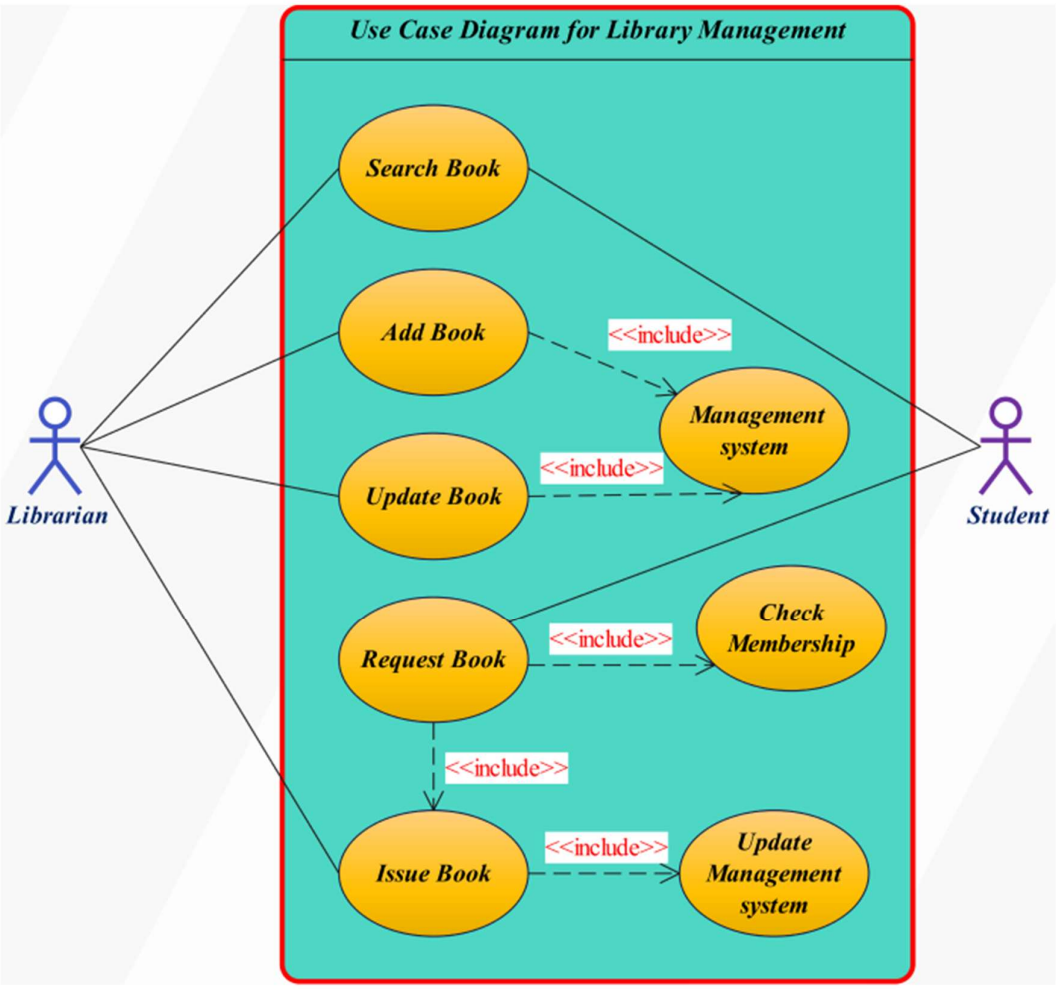
UNIT 2

Library Management System

1. **Use case Diagram**
   Shows what users can do with the system.
   Actors:
   - Member / Student
   - Librarian
   - Admin

   Use Cases:

   - Search Book
   - Issue Book
   - Return Book
   - Renew Book

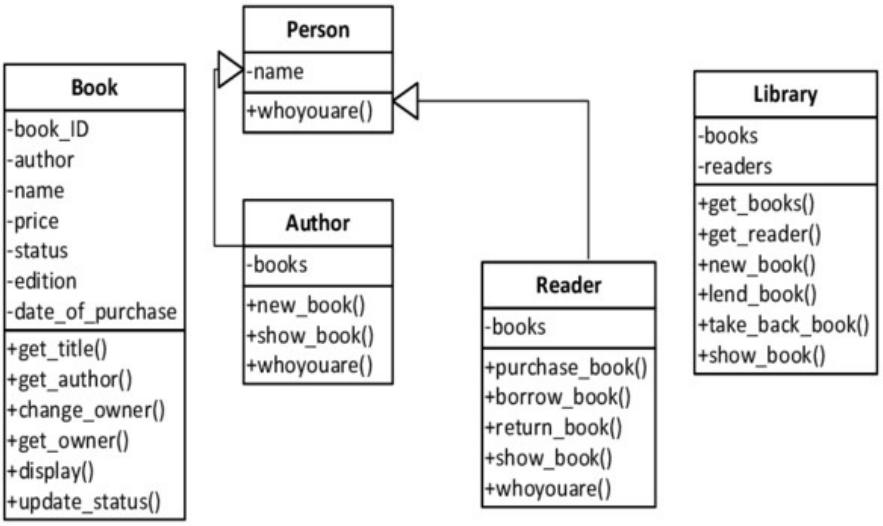


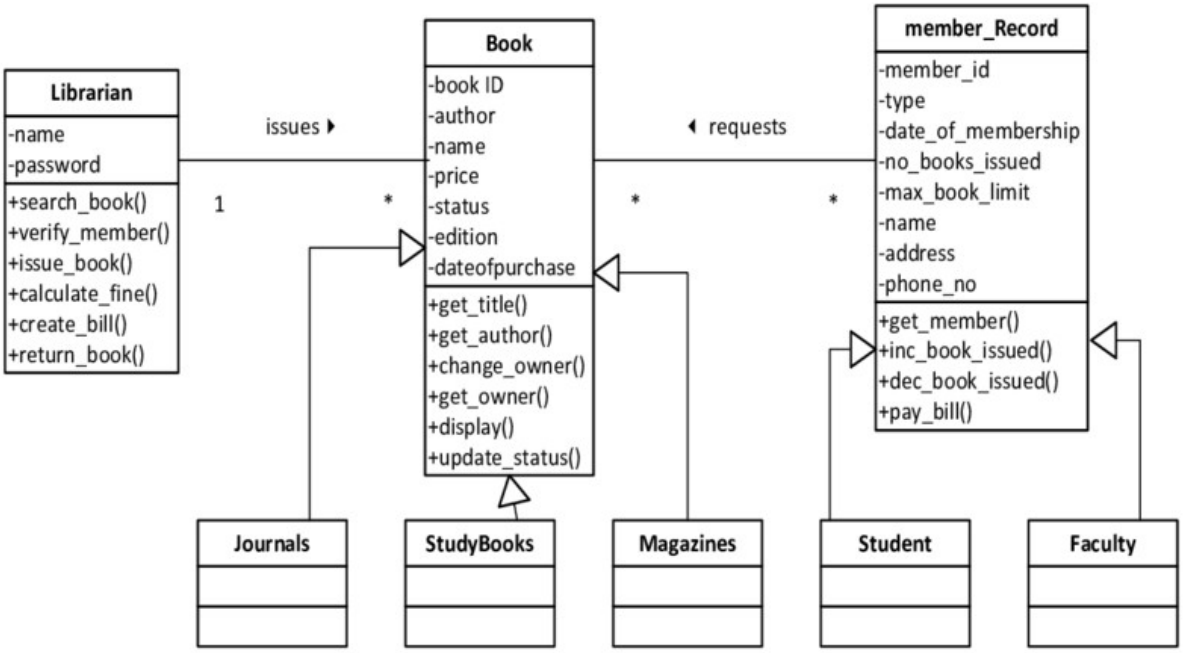
2. Class Diagram

A **Class Diagram** is a structural UML diagram that represents the **static architecture** of a system. It shows:

- Classes
- Attributes
- Operations (methods)
- Relationships such as association, aggregation, composition, and inheritance.

  In a **Library Management System (LMS)**, the class diagram models how books, members, librarians, and transactions are organized and related.



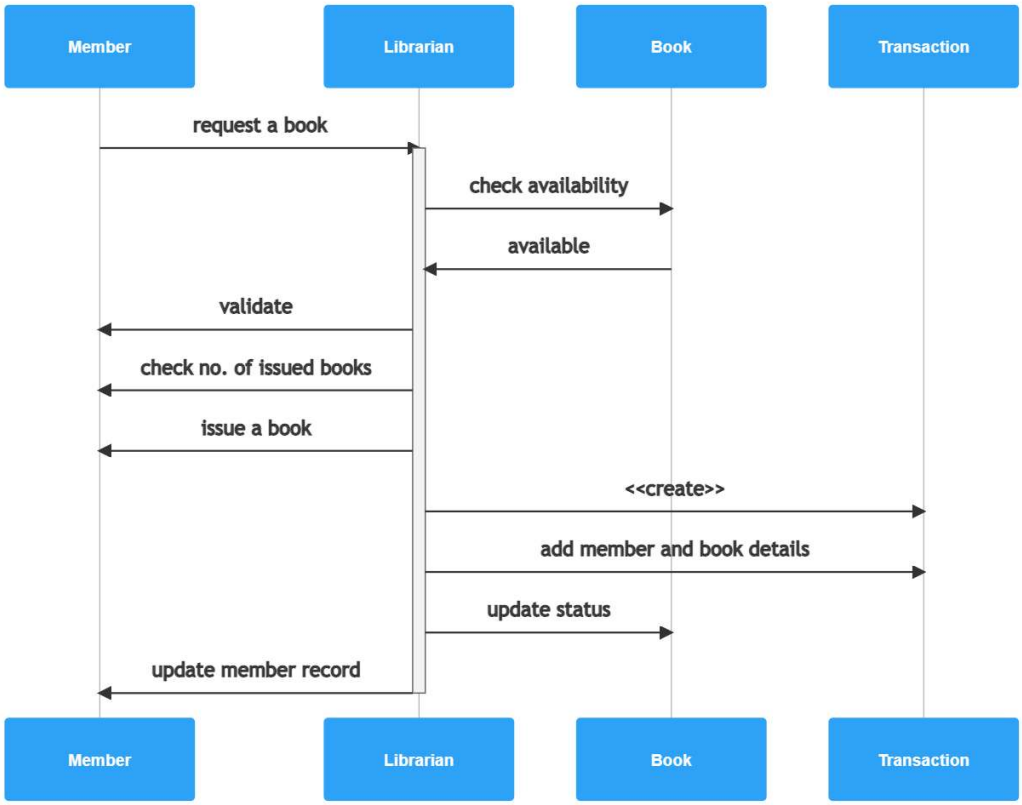


5. **Sequence Diagram**

   Shows **message flow over time**.

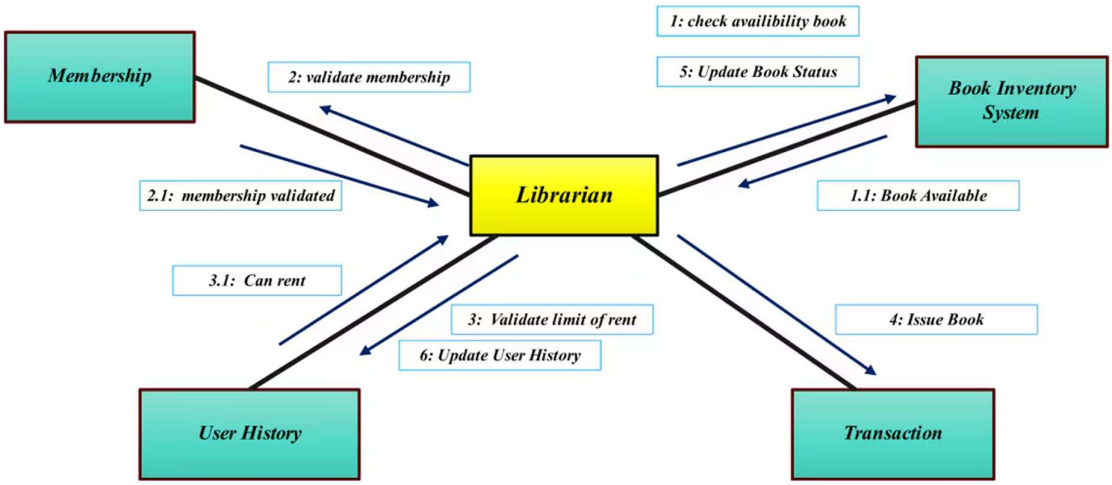
   Example: *Issue Book*

   1. Member → System: requestBook()
   2. System → Catalog: checkAvailability()
   3. Catalog → System: available



6. **Communication diagram**

   Shows **object communication** and links.



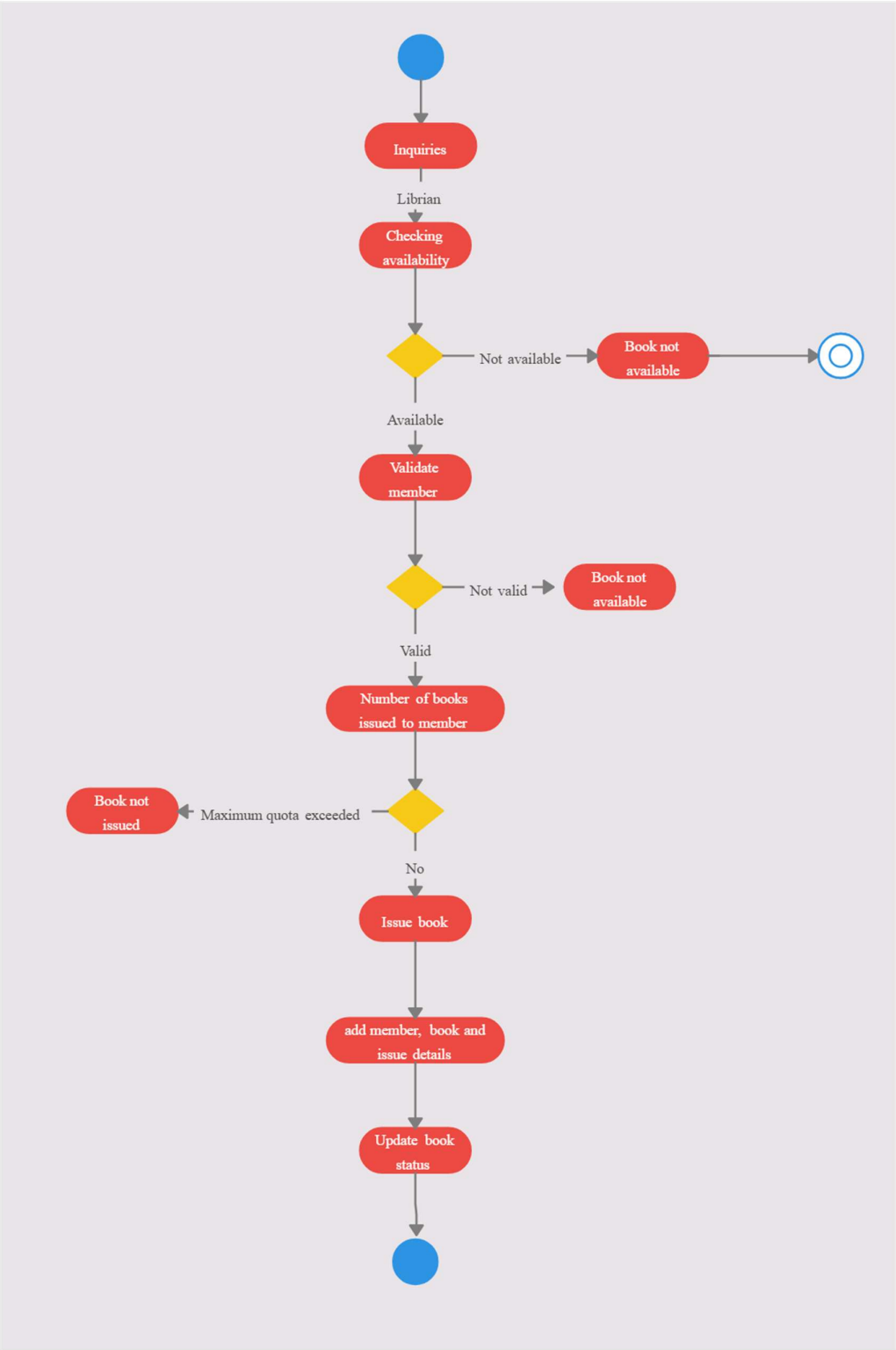
7. **Activity Diagram**

   Models **step-by-step flow of actions**.

   🛠 **Example: Return Book**

   Start → Receive Book → Check Due Date → Is Late?

   - Yes → Calculate Fine → Pay Fine
   - No → Update Record



UNIT -3

1. Software Design Process:

- Software design is an important step during the software development process.
- In software design, we translate the requirements into finished software product in which the details about software data structures, architecture, interfaces and components that are necessary to implement the software system are given.

  Software Design Concepts:

  1. Abstraction
  2. Modularity
  3. Refactoring
  4. Functional Independence
  5. Cohesion
  6. Coupling

1. Abstraction:

- Abstraction refers to the process of showing only the essential features and hiding non-essential details.
- It is used to make the complex systems easier to manage and understand.

2. Modularity:

- Modularity refers to the degree to which a system of independent modules that can be developed, tested and maintained individually.
- It promotes flexibility, reusability and scalability.

3. Refactoring:

- Refactoring is the process of changing a software system in such a way that the external behaviour of the design does not get changed.
- It improves the internal structure of a software system without changing the external design.

4. Functional independence:

- Functional independence refers to the degree to which a module can perform a single, well-defined task without much interaction with other modules.
- It is a key to good design and software quality.
- It helps to improve the modularity of the software design.

5. Cohesion:

- Cohesion is a measure of how well the elements within a module work together to perform a task without interaction with other modules.
- Cohesion helps in information hiding.
- High cohesion indicates the module is focused and efficient.

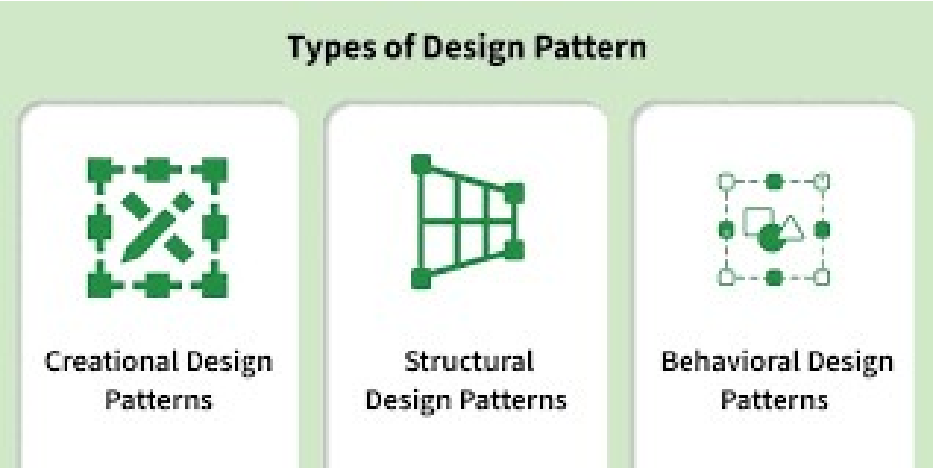
6. Coupling:

- Coupling is a measure of interconnection among modules in a program structure.
- It depends on the interface complexity between the modules.
- Low coupling indicates the modules are relatively independent, making the system more modular and flexible.

---

2. Design Pattern

A design pattern is a general and reusable solution to a commonly occurring problem in software design.



1) Creational Patterns

Deal with object creation in a flexible and controlled way.

Examples:

- **Singleton**
- **Factory Method**
- **Abstract Factory**
- **Builder**
- **Prototype**

2) Structural Patterns

Deal with how classes and objects are connected to form larger systems.

Examples:

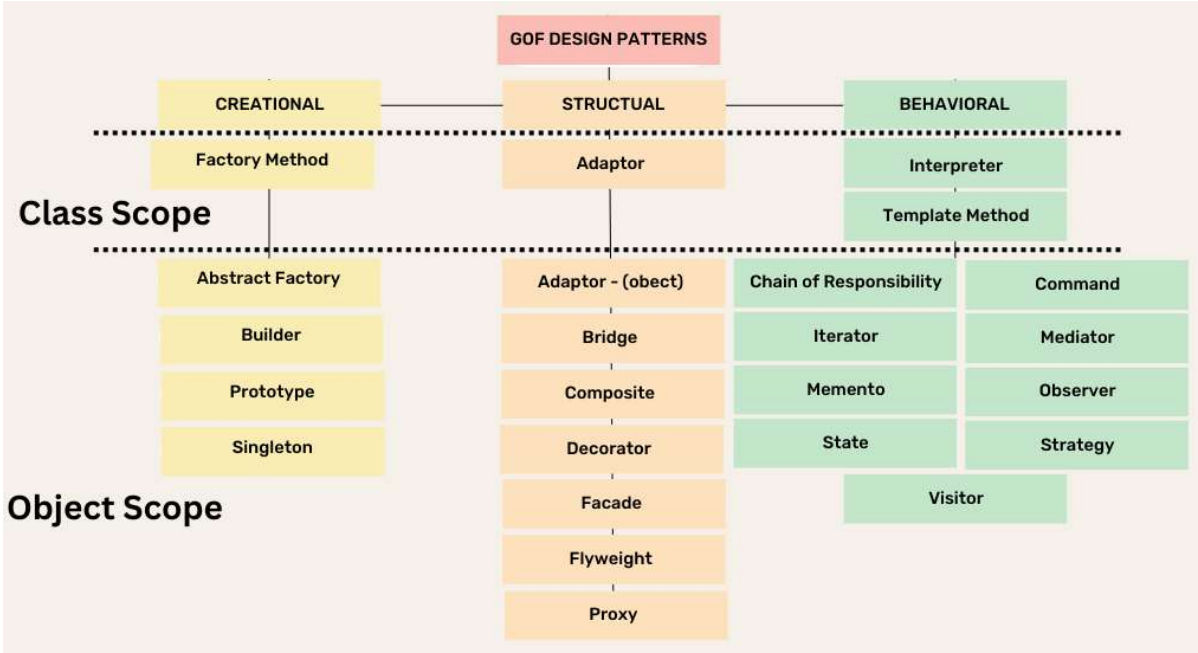
- **Adapter**
- **Bridge**
- **Composite**
- **Decorator**
- **Facade**
- **Proxy**

3) Behavioural Patterns

Deal with communication and responsibility between objects.

Examples:

- **Observer**
- **Strategy**
- **Command**
- **Iterator**
- **Mediator**
- **State**


GOF DESIGN PATTERNS
CREATIONAL
STRUCTUAL
BEHAVIORAL
Factory Method
Adaptor
Interpreter
Class Scope
Template Method
Abstract Factory
Adaptor - (obect)
Chain of Responsibility
Command
Builder
Bridge
Iterator
Mediator
Prototype
Composite
Memento
Observer
Singleton
Decorator
State
Strategy
Object Scope
Facade
Visitor
Flyweight
Proxy